

Low Level Software Working

Student Name:

Minh Quan Tran

Susheela Narayan Hegde

Team Name:

Bin There, **Done** That

Project Name:

Smart parcel sorting system



**SMART PARCEL
SORTING SYSTEM**

Contents

Change logs	4
Week 1 – Initial Setup Raspberry Pi 4B and AWS MQTT Testing	4
Week 2 – MQTT Subscription and Topic Handling	6
Week 3 – UART Communication Between ESP32 and STM32 and Initial draft of every state machine	8
Week 4 – Initial State Machines code and Motor Control	12
Week 5 – Sensor Integration and Actuator Control	14
Week 6 – Updated Flow: STM32 ↔ Raspberry Pi, SPI Setup	17
Proof of Software Working:	19

Table of Figures

Figure 1: ESP32_WiFi Thing	4
Figure 2: Tested QR code	5
Figure 3: Output test_QR.py	5
Figure 4: Esp32 build and flash code	6
Figure 5: ESP32 serial port debug messages after bootup	6
Figure 6: Week 2's tested QR code	7
Figure 7: Output main.py script	7
Figure 8: data printed on esp32/subtopic from Raspberry pi	7
Figure 9: STM32 UART console connected to ESP32	8
Figure 10: ESP32's state machine	9
Figure 11: ESP32's state machine functionalities	9
Figure 12: STM32's state machine	10
Figure 13: STM32's state machine functionalities	10
Figure 14: Rasp 4's state machine	11
Figure 15: output of subscriber.py	12
Figure 16: Idle state (Waiting for msg from STM32)	13
Figure 17: QR state (Read QR)	13
Figure 18: Send data's state	13
Figure 19: L298's wiring	14
Figure 20: STM32's connection	15
Figure 21: Sensor's connection	16
Figure 22: Full system wiring connection	16

Figure 23: Raspberry directly connected to STM32 through SPI	17
Figure 24:Updated state machine for Raspberry Pi	18
Figure 25:Updated state machine for STM32	18
Figure 26: RealVNC installed.....	19
Figure 27: Installing Python	20
Figure 28: Testing Camera.....	20
Figure 29: Running Python's code to read QR code	21
Figure 30: Publishing message to AWSIoT.....	21
Figure 31: Proof that the cloud receives the data	21

Change logs

Week 1 – Initial Setup Raspberry Pi 4B and AWS MQTT Testing

Tasks Completed:

- Installed **Arduino IDE**, set up **ESP32 board** and serial port.
- Created **AWS IoT Thing**, downloaded **certificates**, attached policies.
- Wrote and uploaded **C code** to ESP32 for **MQTT publish/subscribe** to AWS.
- Verified **two-way MQTT communication** with AWS from ESP32.
- Successfully set up Raspberry Pi 4B and accessed it remotely via RealVNC.
- Connected and verified the functionality of the Raspberry Pi Camera Module 3.
- Developed and tested a Python script that captures and decodes QR codes using the camera.

Design Notes:

- Verified that both ESP32 and Raspberry Pi can connect independently to AWS IoT Core via MQTT.
- Certificates and policies handled manually and securely stored.

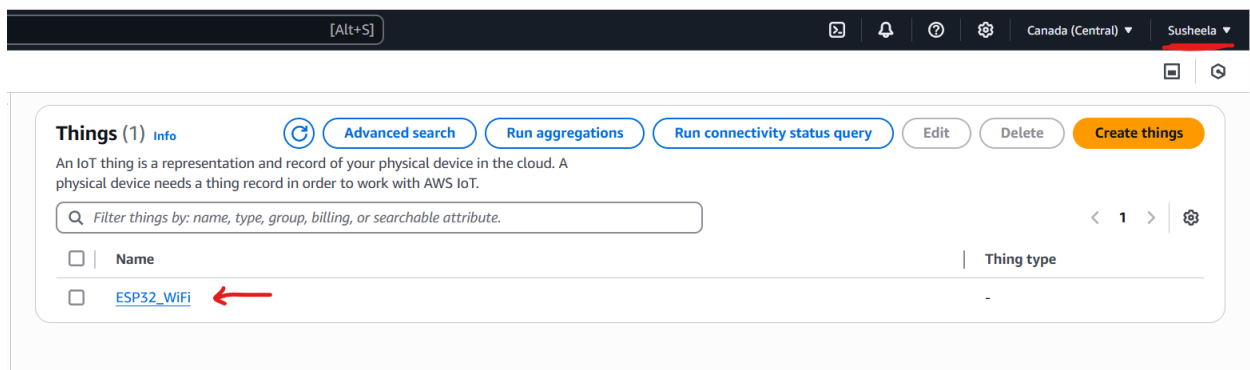


Figure 1: ESP32_WiFi Thing



Figure 2: Tested QR code

```
ESD_Group1@ESDGroup1:~$ python3 test_QR.py
[0:28:39.977479878] [12173] INFO Camera camera_manager.cpp:326 libcamera v0.5.0+59-d83ff0a4
[0:28:40.005779329] [12177] WARN RPiSdn sdn.cpp:40 Using legacy SDN tuning - please consider moving SDN inside rpi.deno
ise
[0:28:40.007763900] [12177] INFO RPI vc4.cpp:447 Registered camera /base/soc/i2c0mux/i2c@1/imx708@1a to Unicam device /
dev/media4 and ISP device /dev/medial
[0:28:40.007876746] [12177] INFO RPI pipeline_base.cpp:1121 Using configuration file '/usr/share/libcamera/pipeline/rpi
/vc4/rpi_apps.yaml'
[0:28:40.014300103] [12173] INFO Camera camera.cpp:1205 configuring streams: (0) 640x480-RGB888 (1) 1536x864-SBGGR10_CS
I2P
[0:28:40.014784395] [12177] INFO RPI vc4.cpp:622 Sensor: /base/soc/i2c0mux/i2c@1/imx708@1a - Selected sensor format: 15
36x864-SBGGR10_1X10 - Selected unicam format: 1536x864-pBAA
Starting QR code scan (up to 10 attempts)...
Attempt 1...
? QR Code Detected: blue
ESD_Group1@ESDGroup1:~$
```

Figure 3: Output test_QR.py

Week 2 – MQTT Subscription and Topic Handling

Tasks Completed:

- Wrote **C++ code in Arduino IDE** to subscribe to MQTT topics.
- Successfully printed received MQTT messages on **ESP32 serial monitor**.
- Understood AWS IoT **policy permissions**: Connect, Subscribe, Publish, Receive.
- Successfully expanding Raspberry Pi Python code to both read the QR data and publish to AWS IoT topics.

Design Notes:

- Validated real-time message reception.
- Started considering **multi-topic architecture** for device coordination.

```
Sketch uses 996478 bytes (76%) of program storage space. Maximum is 1310720 bytes.
Global variables use 46976 bytes (14%) of dynamic memory, leaving 280704 bytes for local variables. Maximum is 327680 bytes.
esptool.py v4.8.1
Serial port COM4
Connecting....
Chip is ESP32-D0WD-V3 (revision v3.1)
Features: WiFi, BT, Dual Core, 240MHz, VRef calibration in efuse, Coding Scheme None
Crystal is 40MHz
MAC: 08:a6:f7:22:cd:d4
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Flash will be erased from 0x00001000 to 0x00007fff...
Flash will be erased from 0x00008000 to 0x00008fff...
Flash will be erased from 0x0000e000 to 0x0000ffff...
Flash will be erased from 0x00010000 to 0x00103fff...
Compressed 24976 bytes to 15972...
Writing at 0x00001000... (100 %)
Wrote 24976 bytes (15972 compressed) at 0x00001000 in 0.6 seconds (effective 338.9 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 146...
Writing at 0x00008000... (100 %)
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.1 seconds (effective 420.4 kbit/s)...
Hash of data verified.
Compressed 8192 bytes to 47...
```

Figure 4: Esp32 build and flash code

```
load:0x40078000,len:16516
load:0x40080400,len:4
load:0x40080404,len:3476
entry 0x400805b4
Connecting to WiFi.....connected
Connecting to AWS IoT...connected
incoming: esp32/sub
blue
```

Figure 5: ESP32 serial port debug messages after bootup



Figure 6: Week 2's tested QR code

```
ESD_Group1@ESDGroup1: ~/Capstone/Integration_QR_AWSIoT $ python3 main.py
[0:03:26.889409096] [2056] INFO Camera camera_manager.cpp:326 libcamera v0.5.0+59-d83ff0a4
[0:03:26.917322109] [2060] WARN RPISdn sdn.cpp:40 Using legacy SDN tuning - please consider moving SDN inside rpi.denoise
[0:03:26.919573093] [2060] INFO RPI vc4.cpp:447 Registered camera /base/soc/i2c0mux/i2c@1/imx708@1a to Unicam device /dev/media1
and ISP device /dev/media0
[0:03:26.919716871] [2060] INFO RPI pipeline_base.cpp:1121 Using configuration file '/usr/share/libcamera/pipeline/rpi/vc4/rpi_ap
ps.yaml'
[0:03:26.932541497] [2056] INFO Camera camera.cpp:1205 configuring streams: (0) 640x480-RGB888 (1) 1536x864-SBGGRR10_CSI2P
[0:03:26.933163905] [2060] INFO RPI vc4.cpp:622 Sensor: /base/soc/i2c0mux/i2c@1/imx708@1a - Selected sensor format: 1536x864-SBGG
R10_1X10 - Selected unicam format: 1536x864-pBAA
Starting QR code scan (up to 10 attempts)...
Attempt 1...
Attempt 2...
Attempt 3...
Attempt 4...
QR Code Detected: blue
Connecting to AWS IoT at 'aqrm5lymlussz-ats.iot.ca-central-1.amazonaws.com'...
Connected to AWS IoT
Published to topic 'esp32/sub':
{
  "message": "blue"
}
Disconnected from AWS IoT
ESD_Group1@ESDGroup1:~/Capstone/Integration_QR_AWSIoT $
```

Figure 7: Output main.py script

Topic filter [Info](#)
The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.

► Additional configuration

Subscribe

Subscriptions

esp32/sub

♥

×

esp32/sub

Message payload

{
 "message": "Hello from AWS IoT console"
}

► Additional configuration

Publish

▼ esp32/sub

May 25, 2025, 11:35:44 (UTC-0400)

{
 "message": "blue"
}

Figure 8: data printed on esp32/subtopic from Raspberry pi

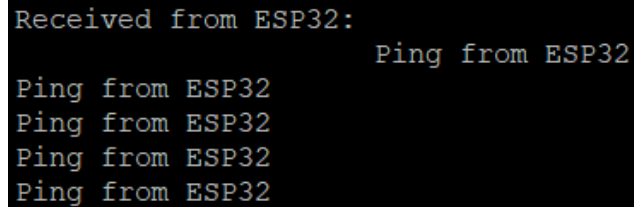
Week 3 – UART Communication Between ESP32 and STM32 and Initial draft of every state machine

Tasks Completed:

- Configured **ESP32 UART1** on GPIO16 (RX) and GPIO17 (TX).
- Connected **ESP32 TX to STM32 RX**, verified signal transmission.
- Wrote **STM32 firmware** to enable UART1, store message in buffer, and print to USB serial terminal.
- Finished drafting the state machine for the whole system.

Design Notes:

- Developed a basic UART driver interface for STM32.
- Validated data integrity from ESP32 to STM32.



```
Received from ESP32:
Ping from ESP32
Ping from ESP32
Ping from ESP32
Ping from ESP32
```

Figure 9: STM32 UART console connected to ESP32

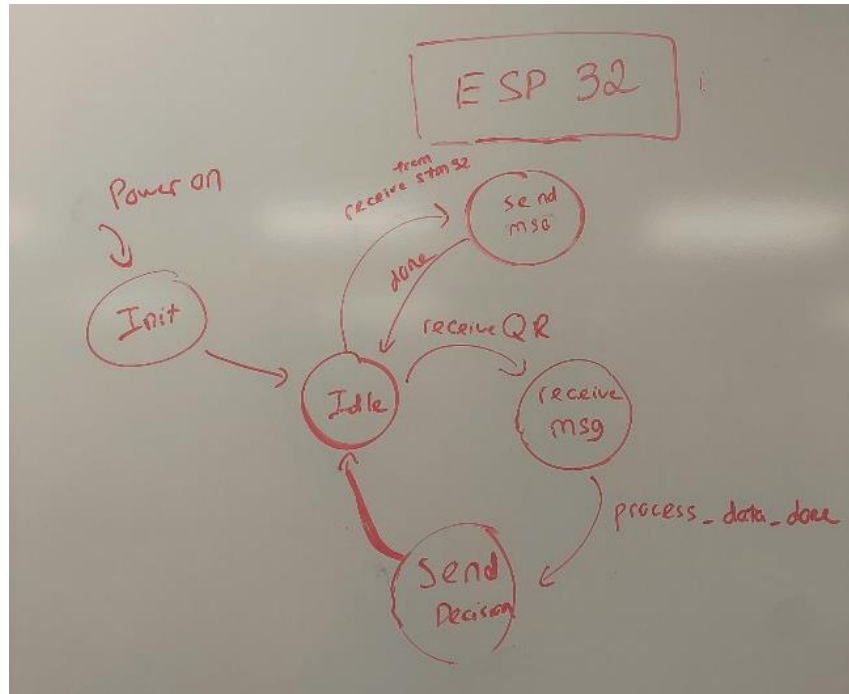


Figure 10: ESP32's state machine

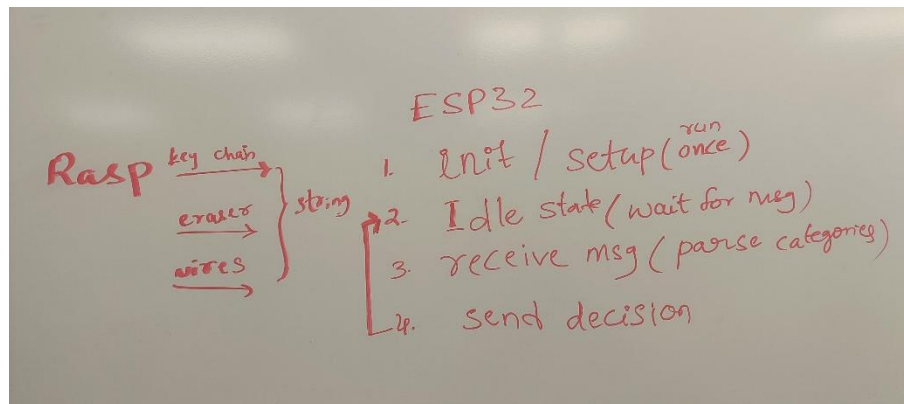


Figure 11: ESP32's state machine functionalities

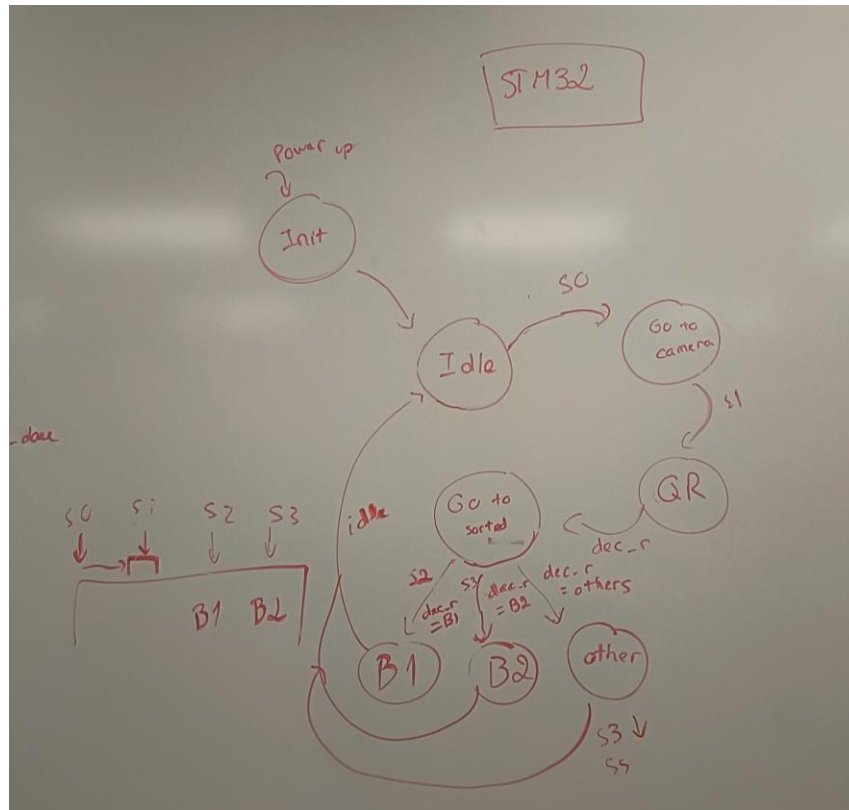


Figure 12: STM32's state machine

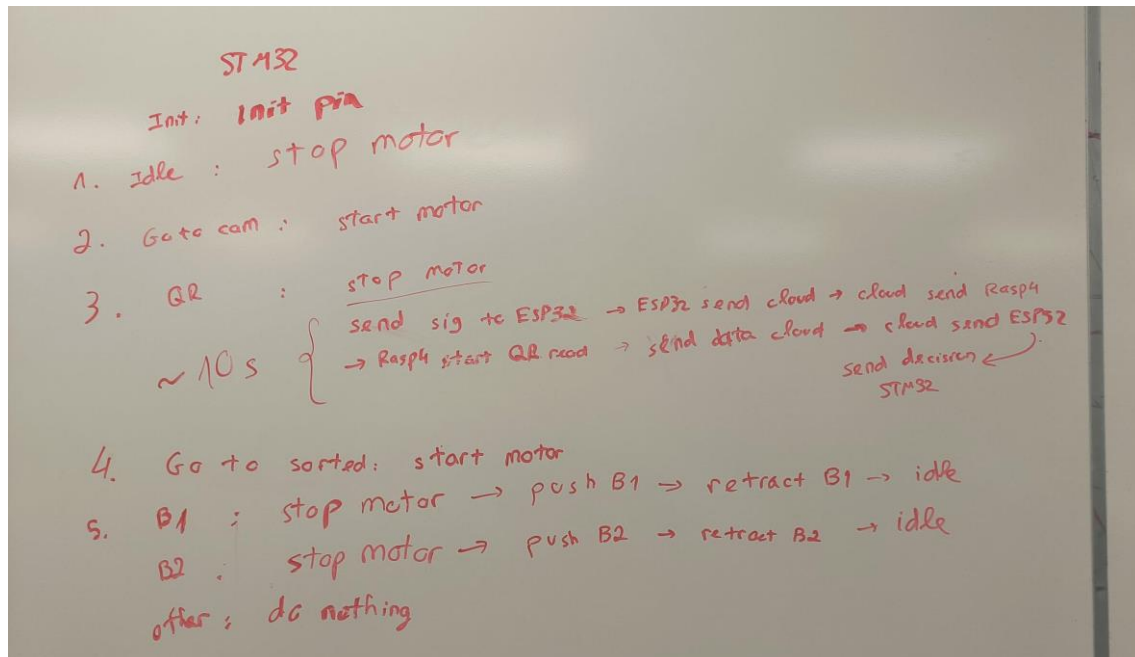


Figure 13: STM32's state machine functionalities

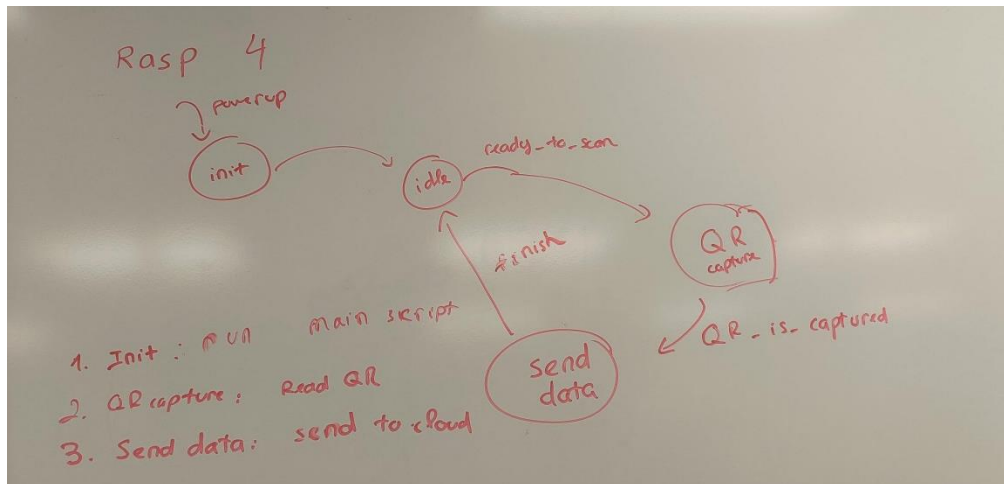


Figure 14: Rasp 4's state machine

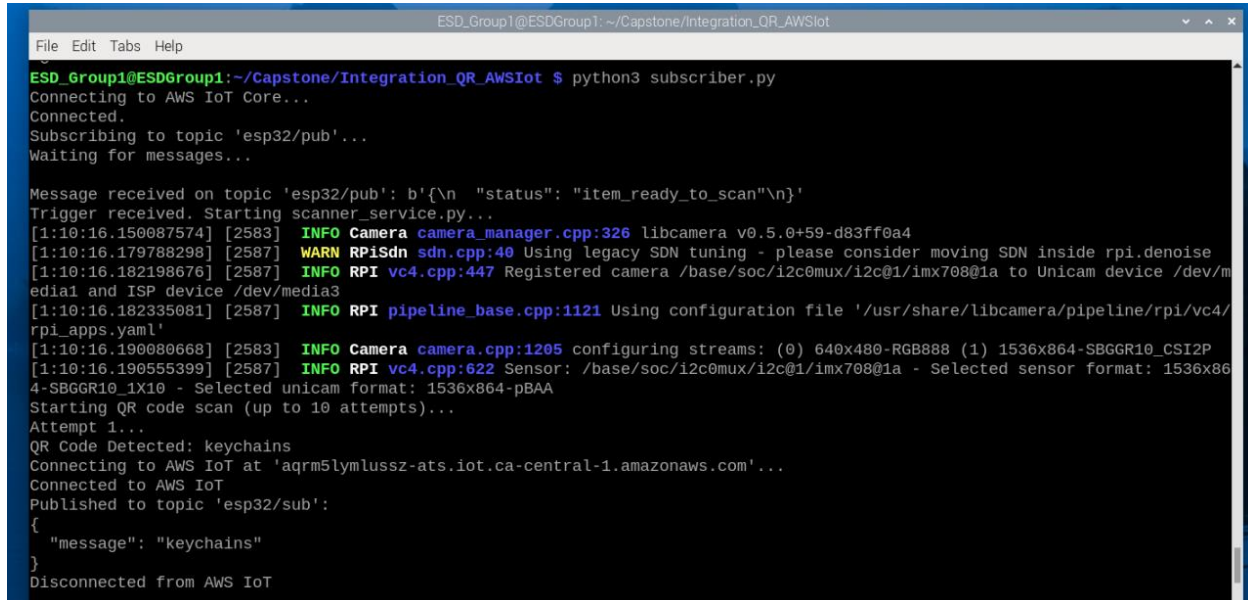
Week 4 – Initial State Machines code and Motor Control

Tasks Completed:

- Developed **initial state machine code** for both ESP32 and STM32.
- Created and tested **DC motor driver code** using **L298N module**.
- Verified motor movement based on control input from code.
- Successfully write and test wiring and coding to control the DC motor in the conveyor.
- Expand the Raspberry pi 4B to work as a state machine.
 - “Idle” state (waiting for esp32_pub signal)
 - “QR” state (Read QR data)
 - “Send data” state (Publish QR data to AWSIoT cloud).
 - Back to “Idle” state after publishing data.

Design Notes:

- Motor state transitions integrated into STM32 state machine.
- Initial control logic tied to UART message parsing.



```
ESD_Group1@ESDGroup1: ~/Capstone/integration_QR_AWSIoT
File Edit Tabs Help
ESD_Group1@ESDGroup1:~/Capstone/Integration_QR_AWSIoT $ python3 subscriber.py
Connecting to AWS IoT Core...
Connected.
Subscribing to topic 'esp32/pub'...
Waiting for messages...

Message received on topic 'esp32/pub': b'{\n  "status": "item_ready_to_scan"\n}'
Trigger received. Starting scanner_service.py...
[1:10:16.150087574] [2583] INFO Camera camera_manager.cpp:326 libcamera v0.5.0+59-d83ff0a4
[1:10:16.179788298] [2587] WARN RPISdn sdn.cpp:40 Using legacy SDN tuning - please consider moving SDN inside rpi.denoise
[1:10:16.182198676] [2587] INFO RPI vc4.cpp:447 Registered camera /base/soc/i2c0mux/i2c@1/imx708@1a to Unicam device /dev/m
edia1 and ISP device /dev/media3
[1:10:16.182335081] [2587] INFO RPI pipeline_base.cpp:1121 Using configuration file '/usr/share/libcamera/pipeline/rpi/vc4/
rpi_apps.yaml'
[1:10:16.190080668] [2583] INFO Camera camera.cpp:1205 configuring streams: (0) 640x480-RGB888 (1) 1536x864-SBGGR10_CSI2P
[1:10:16.190553399] [2587] INFO RPI vc4.cpp:622 Sensor: /base/soc/i2c0mux/i2c@1/imx708@1a - Selected sensor format: 1536x86
4-SBGGR10_1X10 - Selected unicam format: 1536x864-pBAA
Starting QR code scan (up to 10 attempts)...
Attempt 1...
QR Code Detected: keychains
Connecting to AWS IoT at 'aqrm5lymlussz-ats.iot.ca-central-1.amazonaws.com'...
Connected to AWS IoT
Published to topic 'esp32/sub':
{
  "message": "keychains"
}
Disconnected from AWS IoT
```

Figure 15: output of subscriber.py

```
ESD_Group1@ESDGroup1: ~/Capstone/Integration_QR_AWSIoT
File Edit Tabs Help
ESD_Group1@ESDGroup1:~/Capstone/Integration_QR_AWSIoT $ python3 subscriber.py
Connecting to AWS IoT Core...
Connected.
Subscribing to topic 'esp32/pub'...
Waiting for messages...
```

Figure 16: Idle state (Waiting for msg from STM32)

```
Message received on topic 'esp32/pub': b'{\n  "status": "item_ready_to_scan"\n}'
Trigger received. Starting scanner_service.py...
[1:10:16.150087574] [2583] INFO Camera camera_manager.cpp:326 libcamera v0.5.0+59-d83ff0a4
[1:10:16.179788298] [2587] WARN RPISdn sdn.cpp:40 Using legacy SDN tuning - please consider moving SDN inside rpi.denoise
[1:10:16.182198676] [2587] INFO RPI vc4.cpp:447 Registered camera /base/soc/i2c0mux/i2c@1/imx708@1a to Unicam device /dev/m
edial and ISP device /dev/media3
[1:10:16.182335081] [2587] INFO RPI pipeline_base.cpp:1121 Using configuration file '/usr/share/libcamera/pipeline/rpi/vc4/
rpi_apps.yaml'
[1:10:16.190080668] [2583] INFO Camera camera.cpp:1205 configuring streams: (0) 640x480-RGB888 (1) 1536x864-SBGGR10_CSI2P
[1:10:16.190553399] [2587] INFO RPI vc4.cpp:622 Sensor: /base/soc/i2c0mux/i2c@1/imx708@1a - Selected sensor format: 1536x86
4-SBGGR10_1X10 - Selected unicam format: 1536x864-pBAA
Starting QR code scan (up to 10 attempts)...
Attempt 1...
QR Code Detected: keychains
```

Figure 17: QR state (Read QR)

```
QR Code Detected: keychains
Connecting to AWS IoT at 'aqrm5lymlussz-ats.iot.ca-central-1.amazonaws.com'...
Connected to AWS IoT
Published to topic 'esp32/sub':
{
  "message": "keychains"
}
Disconnected from AWS IoT
```

Figure 18: Send data's state

Week 5 – Sensor Integration and Actuator Control

Tasks Completed:

- Wrote **sensor reading code** on STM32 (likely using GPIO/ADC).
- Developed forward and reverse logic for **linear actuator**.
- Finalized **GPIO pin assignments** for all STM32 peripherals.
- Test and confirm all components are in working condition.
- Wiring and successfully run test when all components are connected to STM32.

Design Notes:

- Emphasis on accurate pin mapping for later PCB integration.
- Validated individual actuator behaviors using PWM and direction control.

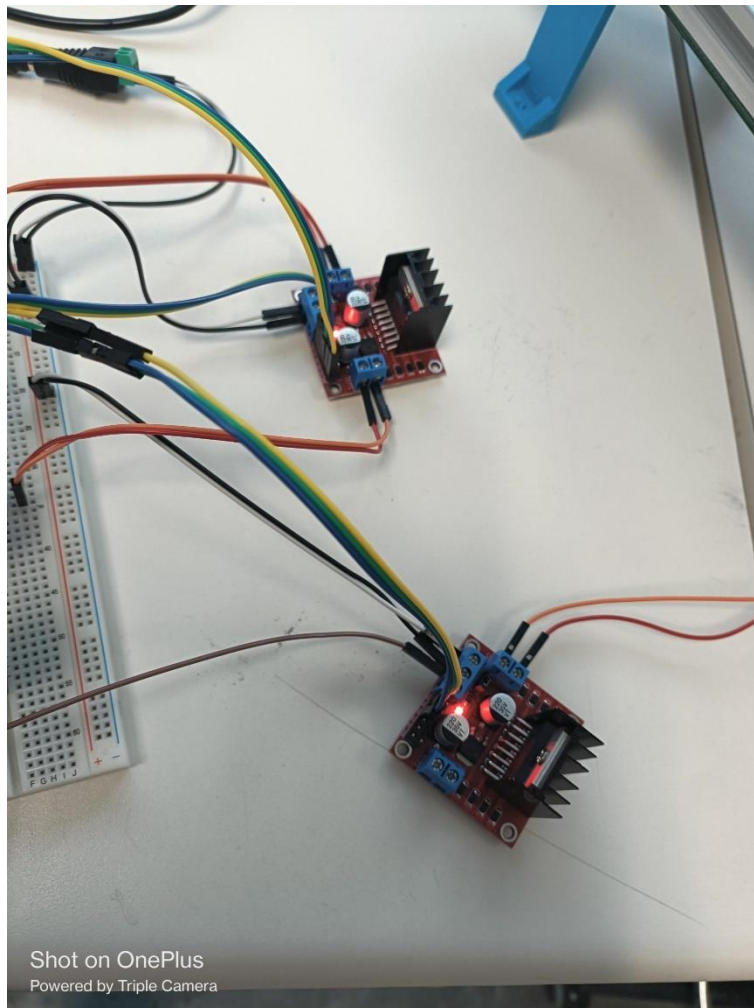


Figure 19: L298's wiring

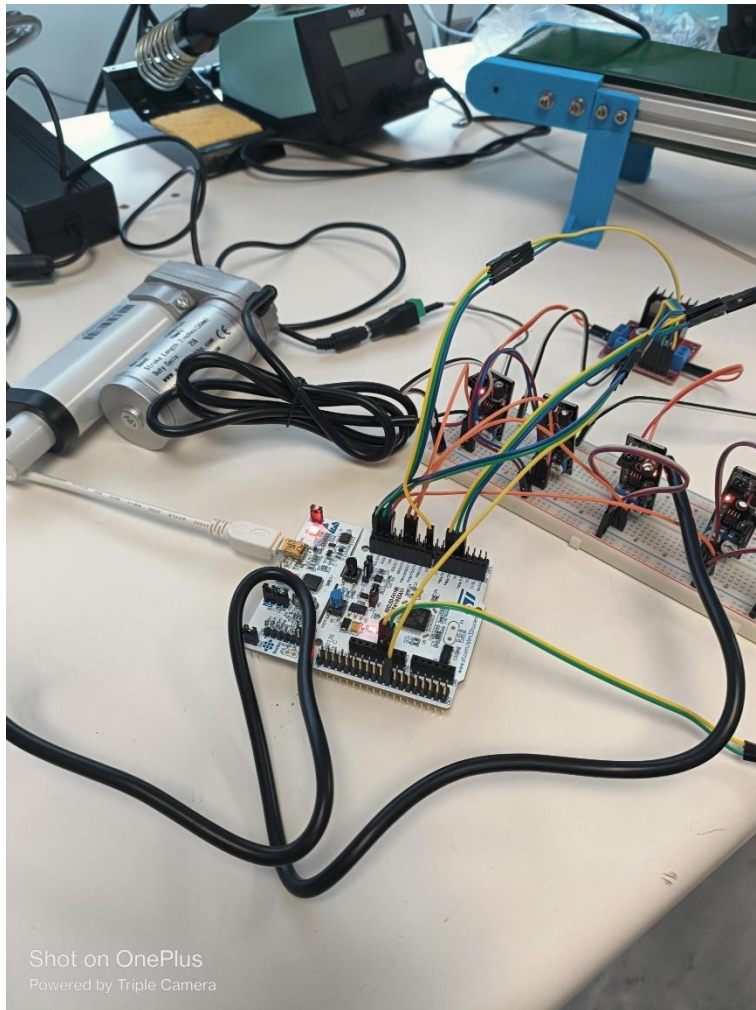


Figure 20: STM32's connection

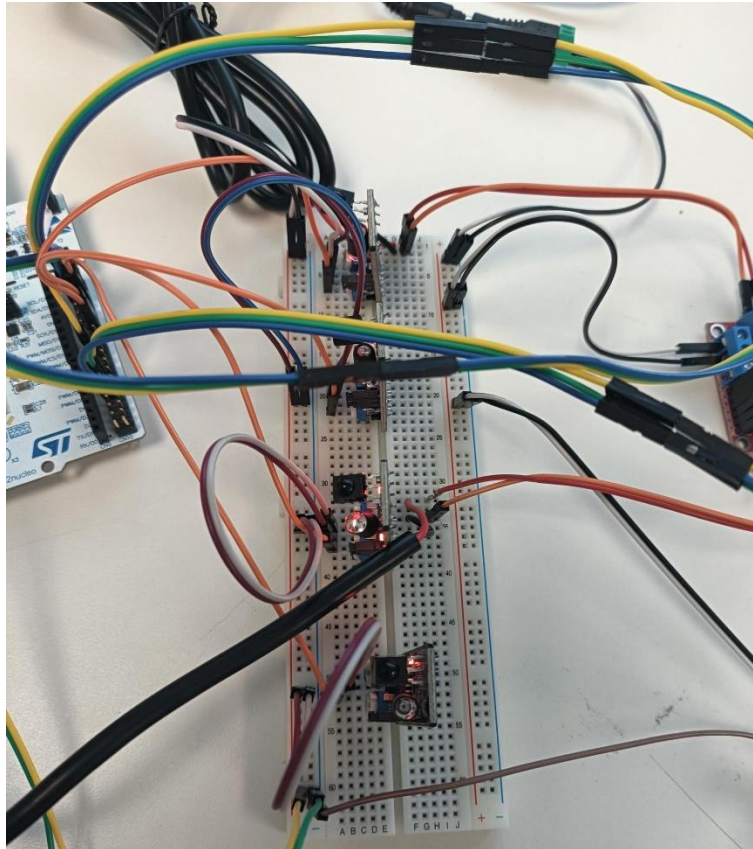


Figure 21: Sensor's connection

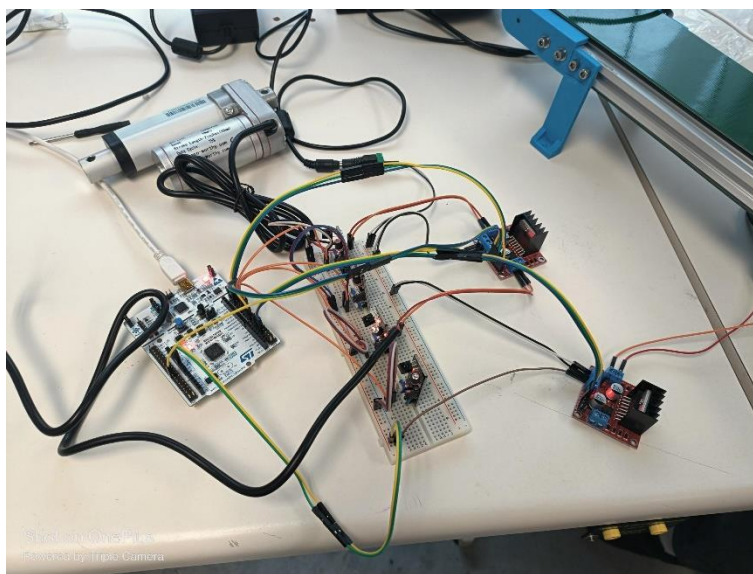


Figure 22: Full system wiring connection

Week 6 – Updated Flow: STM32 ↔ Raspberry Pi, SPI Setup

Tasks Completed:

- Updated state machine logic for **direct STM32-Raspberry Pi communication** (ESP32 now used only for AWS logging).
- Wrote **bidirectional messaging code** over UART between Raspberry Pi and STM32.
- Selected and configured **SPI pins** for inter-device communication.

Design Notes:

- Created **state machine diagrams** to reflect the new system architecture.
- Streamlined ESP32 role to AWS logging only.

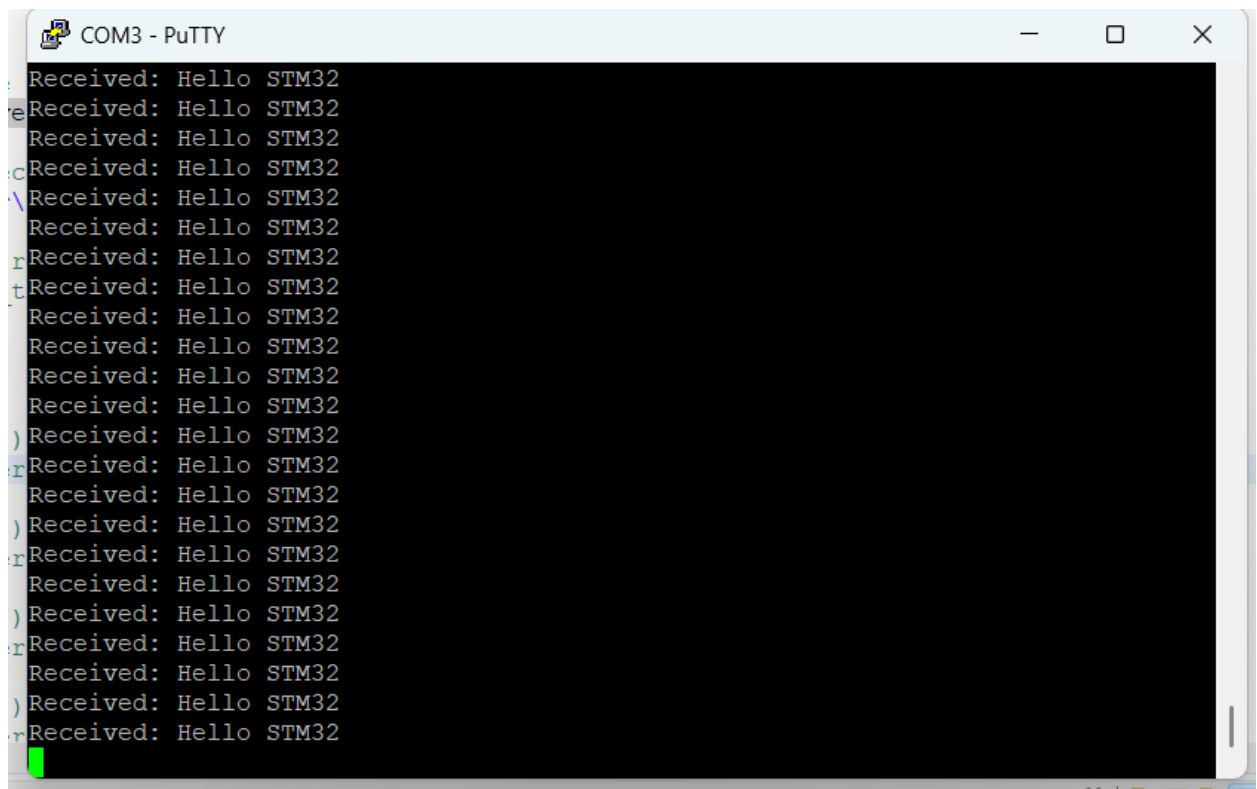


Figure 23: Raspberry directly connected to STM32 through SPI

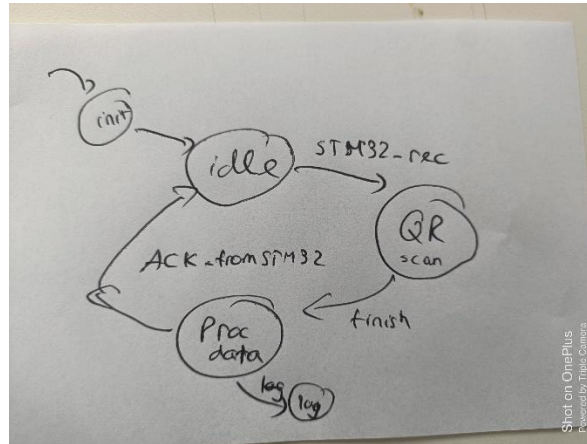


Figure 24: Updated state machine for Raspberry Pi

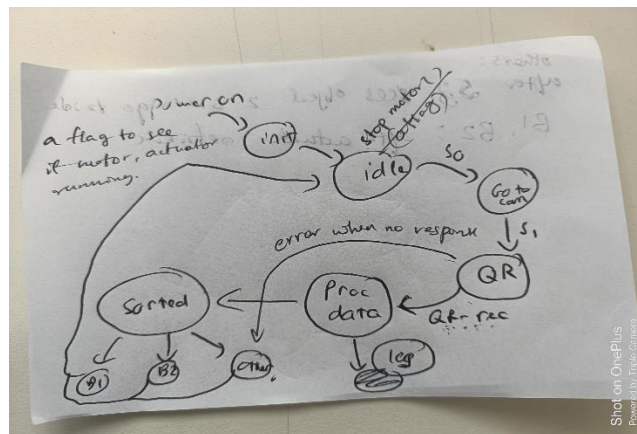


Figure 25: Updated state machine for STM32

Screenshots and video links:

1. DC Motor, Conveyor working : [video link](#)
2. All the sensors, Actuator and Conveyor working: [video link](#)

Proof of Software Working:

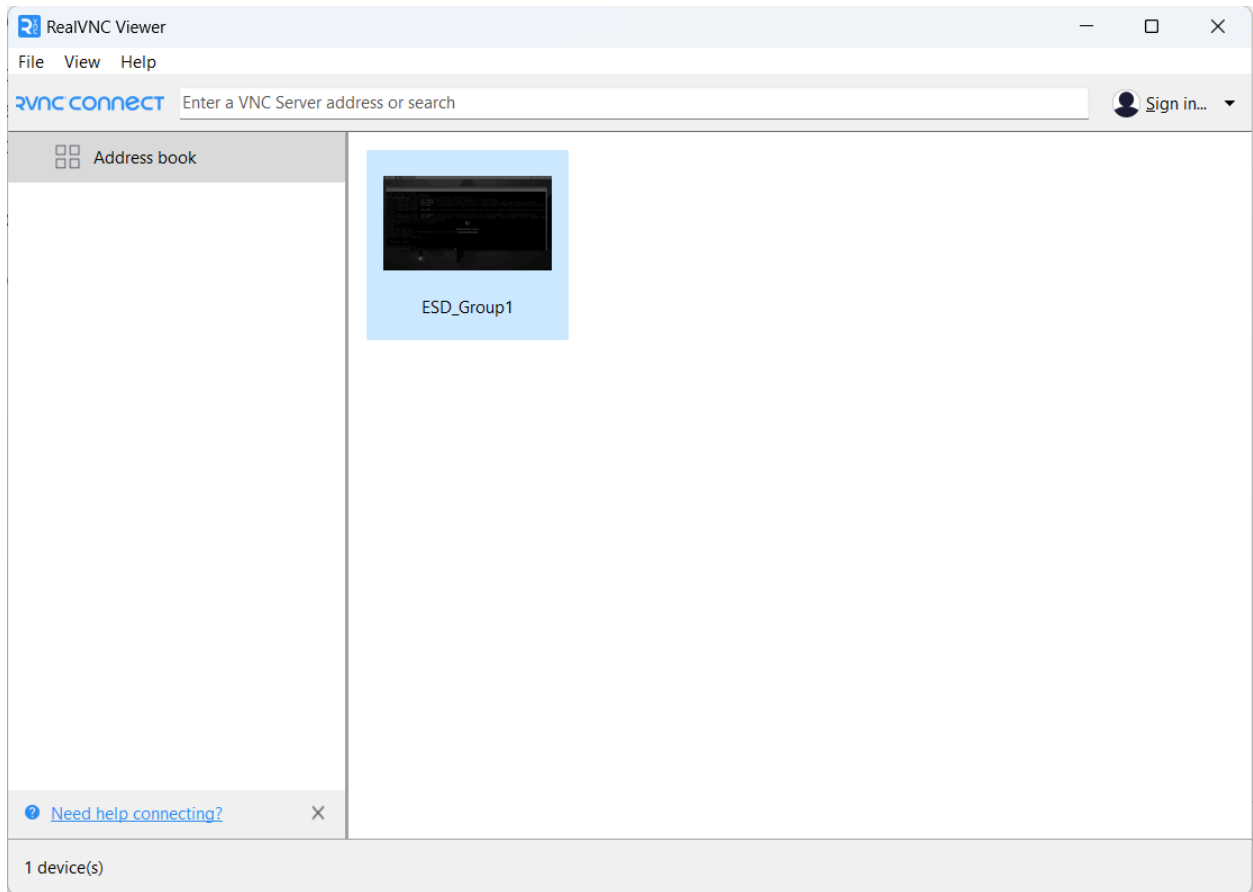
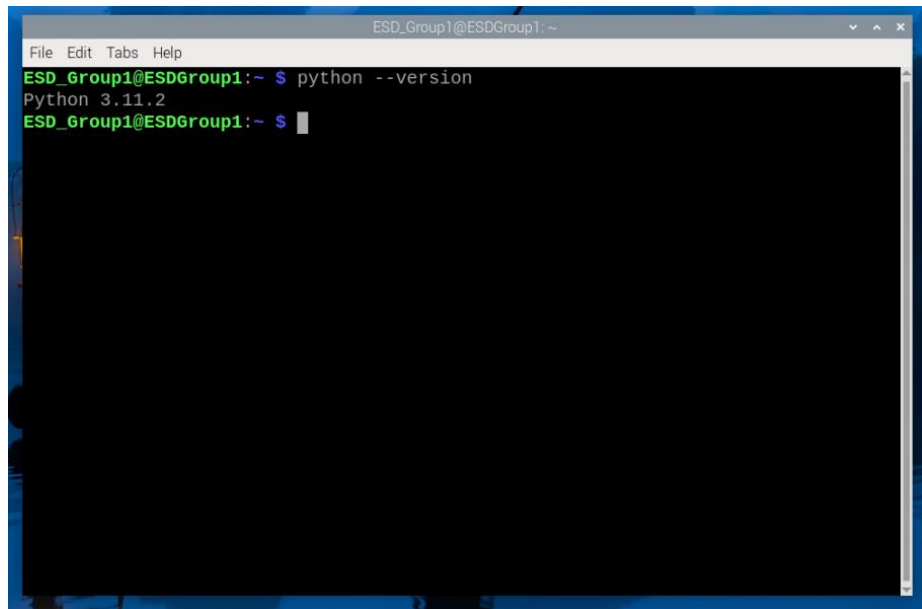


Figure 26: RealVNC installed

A terminal window titled 'ESD_Group1@ESDGroup1: ~' with a menu bar (File, Edit, Tabs, Help). The prompt is 'ESD_Group1@ESDGroup1:~'. The user enters 'python --version' and the output is 'Python 3.11.2'. The prompt returns to 'ESD_Group1@ESDGroup1:~' with a cursor.

```
ESD_Group1@ESDGroup1:~ $ python --version
Python 3.11.2
ESD_Group1@ESDGroup1:~ $
```

Figure 27: Installing Python

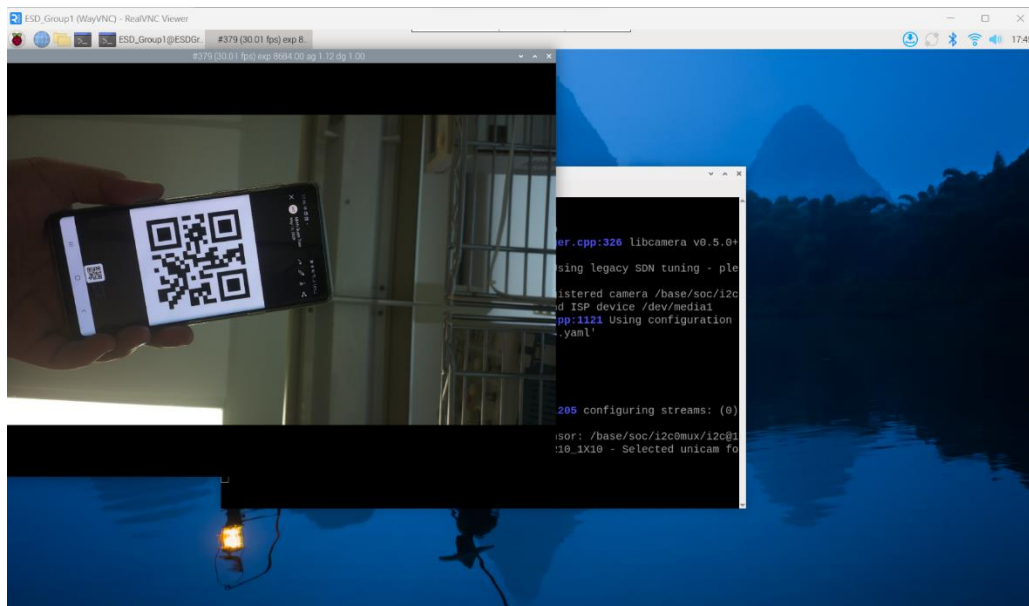


Figure 28: Testing Camera

